CSE/DSC 234 Project 2 Schema Linking with Supervised Fine-Tuning

Fengfei Yu (A69042700)
Team 51

Abstract

This report presents a schema-linking system for natural-language questions over relational databases. The task input is a question and a database schema, and the output is a set of tables and columns that the corresponding SQL query should touch. I trained LoRA adapters on the released labeled examples and evaluated each run with the released validation scorer. The final system uses Qwen2.5-Coder 1.5B and a single multi-task LoRA adapter that is trained, from the same gold links, on five task views per example (tables, full schema-links, columns given the selected tables, clause-specific columns, and a verify decision). At inference one adapter produces four of these views per question and a deterministic vote (keep a table with one view, keep a column with two) combines them, followed by JSON validation. The experiments show a clear progression from a weak no-SFT baseline through a pruned, balanced coder-model adapter to the multi-task formulation. The submitted pipeline scored 0.7315 overall, with a table score of 0.8043 and a column score of 0.6587.

1 Training Data Methodology

The training data contains 301 examples. Each example gives a question, a database identifier, a gold SQL query, and the table and column links extracted from that SQL query. The schema files provide the database structure used by the prompts, including table names, columns, types, primary keys, and foreign-key relationships. During preprocessing, I normalized every target link against these schema files so the output uses the original table and column spelling. I kept the released validation set separate and used it only for scoring and model selection.

Each supervised example becomes one prompt and one JSON target. The prompt contains the task instruction, the question, and a schema view for the database. The target is a dictionary from table names to linked columns, with an empty list for table-only references. During fine-tuning, the question and schema are treated as context. Only the JSON target contributes to the training loss, which makes the optimization match the final prediction task more closely than training the model to reproduce the whole prompt.

The dataset itself is small and uneven across databases. Eight SAP Business One modules have 6 examples each, Human Resources has 12, six standalone databases have 24 each, NYSED has 37, and NTSB has 60. This imbalance is not part of the label definition; it motivated a separate training choice that tests whether mild upsampling of low-resource databases would improve validation performance without discarding larger-database examples.

Multi-task reformulation. The strongest data choice does not add new labels; it re-expresses the same gold links as several task views per example. From one labeled row I build: (1) a tables-only target, (2) the full schema-links target, (3) a columns target conditioned on the already-selected tables, (4) clause-specific column targets obtained by parsing the gold SQL into SELECT, JOIN, WHERE/HAVING, and GROUP/ORDER slices, and (5) a yes/no verify decision on a candidate table.column pair, with hard negatives sampled from the pruned schema. Balancing the 301 rows to 349 and expanding them this way yields 2,893 training rows for the multi-task (“mixed”) adapter. The two-stage variant uses only views (1) and (3) (698 rows). Decomposing the task this way

lets one adapter learn table recall and column precision from several complementary angles while staying a single submitted artifact.

2 Final Pipeline Architecture

The final inference system runs locally and does not call external APIs. It loads the Qwen2.5-Coder-1.5B-Instruct base model and attaches the submitted adapter from the repository. This keeps the deployed artifact small while staying within the project limit of at most 2B base-model parameters.

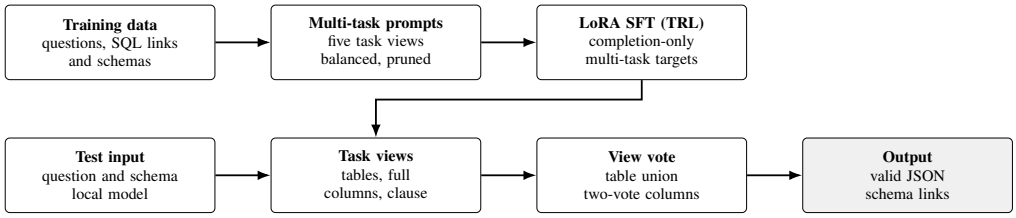

Figure 1: Final pipeline overview. The training path builds one multi-task LoRA adapter from balanced, pruned prompts expressed as five task views. At inference time the same adapter produces four views per question; a table-level union and two-vote column rule combine those predictions before JSON validation.

Model and adapter. The final run uses Qwen2.5-Coder-1.5B-Instruct rather than the general Qwen2.5 1.5B instruction model. Schema linking is close to code and SQL work. The model must preserve exact identifiers, reason about joins and filters, and output a strict object. The coder base model gave better table and column scores under the same data setup [3]. I trained a LoRA adapter instead of updating the full model, which kept the submitted artifact small and made training practical on the available GPU slice [4].

Hyperparameters. The submitted adapter uses rank 16, alpha 32, and LoRA dropout 0.05 on the seven attention and MLP projections. Training used BF16 compute, batch size 1, gradient accumulation 8, cosine scheduling, 5% warmup, maximum length 8192, learning rate 8.5×10^{-5}, and 2 epochs over the multi-task data. Two epochs is a sweet spot: three or four epochs, or rank 32, overtrain and lose table score, and the small column gain does not compensate.

Training engine. The adapter is trained with TRL's `SFTTrainer` directly. In the rebuilt environment (rapidfireai 0.16 with trl 0.21, peft 0.19, transformers 4.57) the RapidFire fit path left the LoRA adapter non-trainable: the gradient norm stayed at 0 and the saved low-rank weights were all zero. The direct trainer produces the same single-adapter artifact, and data preparation, inference, and scoring are unchanged.

Data balancing. Upsampling is a trick in the best run, not part of the original data. I repeated only databases with fewer than 12 examples until they reached 12 rows. This changed the training file from 301 to 349 rows and mainly helped small SAP modules receive enough updates. I did not downsample NTSB or NYSED because their examples are not duplicates; they cover real query and schema patterns.

Prompt building. The final prompt uses a pruned schema view instead of the full database schema. Candidate schema items are selected by question overlap, simple synonym handling, and link statistics from the training split. For training rows, gold links are always included in the prompt so the model is not asked to predict a hidden identifier. For validation and test rows, the same pruning logic runs without gold answers. The default inference view uses 14 tables and 320 columns.

Inference views. The submitted pipeline uses only the adapter in the repository root, but it asks that adapter to predict four task views per question. The tables view names the relevant tables; the full view predicts the whole schema-links object; the columns view predicts columns conditioned on the tables already selected; and the four clause views ask separately for the columns used by the SELECT, JOIN, WHERE/HAVING, and GROUP/ORDER parts of the query. These views are not separate checkpoints; they are different prompts for the same trained adapter. Generation is batched across questions per view, which keeps a full run on about 100 questions near two and a half minutes, well inside the time budget.

Voting rule. A table is kept if it appears in any view (threshold one). A column is kept only if at least two views predict it, and only under a table that passed the table vote (threshold two). The two-vote column rule is the decisive inference knob: it directly raises column precision, the metric that limits the system. The verify view was also available as a yes/no pruning pass, but it over-pruned columns (it cut the column score from about 0.66 to 0.53), so it is disabled in the final pipeline.

Decoding and validation. Generation is greedy and deterministic. The predicted text is parsed as JSON, duplicate columns are collapsed, invalid tables or columns are removed, and valid names are restored to the original schema casing. This is a conservative validity filter. It improves precision by removing identifiers that cannot be scored as correct, but it does not add missing tables or columns.

3 Experimentation Methodology

I compared supervised fine-tuning configurations under a fixed evaluation loop: train an adapter, generate predictions on the held-out development questions, and score them with the released set-based evaluator. The earlier single-task sweep used the RapidFire AI multi-config runner; the final task-decomposition adapters were trained with TRL directly, for the environment reason noted in Section 2. The table score is the average of macro precision, recall, and F1 over table sets. The column score is computed the same way over table-column pairs, and the overall score averages those two scores.

The search was organized as a sequence of controlled decisions rather than a single random sweep. **First**, I measured the base model without fine-tuning to understand the prompt-only floor. **Second**, I added supervised fine-tuning and compared loss formatting, which made completion-only training the default. **Third**, I changed the schema view from full schema to pruned schema because the main failure pattern was extra columns. **Fourth**, I tested mild database balancing because several databases had only 6 examples. **Fifth**, I switched to a coder base model and tuned the learning rate, epoch count, and adapter size. **Finally**, I compared two single-adapter task decompositions on top of the coder adapter: a two-stage adapter that predicts tables and then columns, and a multi-task "mixed" adapter trained on all five views, and I tuned the inference vote thresholds and the verify pass. This order made each improvement interpretable because every stage answered one question before moving to the next.

Table 1: Representative configurations on the released validation split. Each row changes a model, data, training, or inference choice.

Run	Base model	Data and schema setting	Training setting	Table	Column	Overall
No SFT	Qwen2.5 1.5B	full schema	no adapter	0.4113	0.2579	0.3346
Completion only	Qwen2.5 1.5B	full schema	rank 16, alpha 16, 2 epochs, lr $2e{-}4$	0.6523	0.4729	0.5626
Pruned schema	Qwen2.5 1.5B	pruned schema	rank 16, alpha 16, 4 epochs, lr $2e{-}4$	0.7049	0.5644	0.6347
Balanced to 12	Qwen2.5 1.5B	pruned, database target 12	rank 16, alpha 16, 4 epochs	0.7254	0.5819	0.6536
Coder model	Qwen2.5-Coder 1.5B	pruned, database target 12	rank 16, alpha 32, 6 epochs, lr $1e{-}4$	0.7524	0.5945	0.6735
Best single adapter	Qwen2.5-Coder 1.5B	pruned, database target 12	7 epochs, lr $8.5e{-}5$	0.7741	0.6156	0.6948
Two-stage adapter	Qwen2.5-Coder 1.5B	tables-then-columns views	rank 32, alpha 64, 7 epochs	0.7667	0.6707	0.7187
Mixed, verifier on	Qwen2.5-Coder 1.5B	five task views	rank 16, alpha 32, 2 epochs	0.8045	0.5810	0.6928
Final mixed vote	Qwen2.5-Coder 1.5B	four task views, votes $(1, 2)$	same adapter, verifier off	**0.8043**	**0.6587**	**0.7315**

Table 2: Metric breakdown for the main improvement path. Table and column scores are each the average of their precision, recall, and F1 columns.

Run	P_T	R_T	$F1_T$	Table	P_C	R_C	$F1_C$	Column	Overall
No SFT	0.3230	0.5553	0.3556	0.4113	0.3079	0.2479	0.2180	0.2579	0.3346
Completion only	0.6832	0.6361	0.6375	0.6523	0.5066	0.4632	0.4488	0.4729	0.5626
Pruned schema	0.7153	0.7112	0.6882	0.7049	0.5845	0.5688	0.5400	0.5644	0.6347
Balanced to 12	0.7231	0.7450	0.7079	0.7254	0.5921	0.5978	0.5558	0.5819	0.6536
Coder model	0.7607	0.7591	0.7375	0.7524	0.6018	0.6092	0.5725	0.5945	0.6735
Best single adapter	0.7795	0.7830	0.7596	0.7741	0.6265	0.6291	0.5910	0.6156	0.6948
Final mixed vote	**0.7973**	**0.8267**	**0.7888**	**0.8043**	**0.6721**	**0.6727**	**0.6313**	**0.6587**	**0.7315**

Tables 1 and 2 show the path to the final run. Each stage fixed a concrete weakness from the previous stage, moving from output format to schema noise, data imbalance, base-model fit, optimization, and then task decomposition. The two-stage adapter already cleared the prior single-adapter result by separating table recall from column selection; the multi-task adapter went further by raising the table

score to 0.80. Heavier balancing, a larger rank, and more epochs all lowered the combined score, so they were not used in the final system.

This experimental path also explains why the final pipeline stays conservative. Full schemas gave the model too many plausible columns to copy, and training the mixed adapter past two epochs overfit the small data and cost table score. The chosen pipeline is therefore not the largest or most complicated one I tried; it is the setup where each component improved the held-out score without a larger regression elsewhere. The multi-task formulation and the two-vote column rule are the changes that carried the combined score past the earlier submission while keeping a single submitted adapter.

4 Results and Analysis

Building the single-adapter baseline. The prompt-only base model scored only 0.3346, with table recall (0.5553) far above table precision (0.3230): it found related tables but returned too many and often emitted invalid JSON or columns under the wrong table. Completion-only fine-tuning was the largest single jump (to 0.5626), because masking the prompt makes every loss-bearing token part of the JSON answer. Pruning the schema (0.6347) then cut the dominant error, plausible but wrong columns, lifting column F1 from 0.4488 to 0.5400. Mild balancing of databases with fewer than 12 examples added recall (0.6536); a stronger target of 18 hurt column score, so balancing stays gentle and never downsamples the large databases. Switching to the Qwen2.5-Coder base (0.6735) was the largest later gain, since exact identifiers and SQL-style reasoning match a code model. A learning-rate and epoch search then settled on 7 epochs at 8.5×10^{-5} for the single-task adapter (0.6948); rank 32 and higher rates regressed, so the limit is stable column selection on a small, uneven dataset rather than adapter capacity.

Effect of the multi-task adapter. Training the two tricks as single adapters separated the strengths. The two-stage adapter (tables then columns) reached 0.7187 and gave the best column precision, because the column step only ever sees already-selected tables. The mixed adapter reached the highest table score, 0.8043, because predicting tables, full links, columns, and clause columns together teaches table identification from several angles. At inference the four-view vote then sets the precision/recall balance: keeping a column only when two views agree lifted column precision and carried the combined score to 0.7315. The verify pass, by contrast, pruned too aggressively and was turned off. Both winning systems are single adapters, so the packaging stays one artifact.

Remaining errors. Table prediction is stronger than column prediction. Many questions reveal the correct table from surface words, but the right column depends on a measure, condition, date field, or join path. The final post-processing can remove invalid identifiers, but it cannot recover a missed measure column or infer a join key that none of the views predicted. In the submitted run, table score was 0.8043 while column score was 0.6587, so most remaining room is still in fine-grained column selection.

5 AI Tools Statement

AI tools were used as coding, debugging, experiment-organization, and writing aids. They helped with inspecting project files, organizing commands, summarizing results for experiment runs, and editing report/README wording. The experiments, scores, and submitted adapter come from local project runs and the released evaluator.

References

[1] Kyle Luoma and Abhinav Kumar. SNAILS: Schema Naming Assessments for Improved LLM-Based SQL Inference. Proc. ACM SIGMOD, 2025.

[2] RapidFire AI. RapidFire AI. `https://www.rapidfire.ai/`.

[3] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, et al. Qwen2.5-Coder technical report. arXiv:2409.12186, 2024.

[4] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, et al. LoRA: Low-rank adaptation of large language models. arXiv:2106.09685, 2021.